LAPORAN PRAKTIKUM

STRUKTUR DATA

MODUL 6

DOUBLY LINKED LIST (BAGIAN PERTAMA)



Disusun oleh:

Salsadilla Hanny Azizah

109082500014

S1IF-13-02

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

TAHUN 2026

LATIHAN SOAL MODUL

Link Repo Github Modul 6 : salsadha/PRAKTIKUMSDT

1. **Soal 1**

Doublylist.h

```
#ifndef DOUBLYLIST_H_INCLUDED
#define DOUBLYLIST_H_INCLUDED
#include <iostream>
#include <string>

#define Nil NULL
using namespace std;

struct kendaraan {
    string nopol;
    string warna;
    int thnBuat;
};

typedef kendaraan infotype;
typedef struct ElmList *address;

struct ElmList {
    infotype info;
    address next;
    address prev;
};

struct List {
    address First;
    address Last;
};

void CreateList(List &L);
address alokasi(infotype x);
void dealokasi(address &P);
void printInfo(List L);
void insertLast(List &L, address P);
address findElm(List L, infotype x);

#endif
```

Doublylist.cpp

```
#include "Doublylist.h"
```

```
void CreateList(List &L) {
    L.First = Nil;
    L.Last = Nil;
}

address alokasi(infotype x) {
    address P = new ElmList;
    if (P != Nil) {
        P->info = x;
        P->next = Nil;
        P->prev = Nil;
    }
    return P;
}

void dealokasi(address &P) {
    delete P;
    P = Nil;
}

void insertLast(List &L, address P) {
    if (L.First == Nil) {
        L.First = P;
        L.Last = P;
    } else {
        P->prev = L.Last;
        L.Last->next = P;
        L.Last = P;
    }
}

void printInfo(List L) {
    address P = L.First;
    cout << "DATA LIST 1\n" << endl;
    while (P != Nil) {
        cout << "no polisi : " << P->info.nopol << endl;
        cout << "warna     : " << P->info.warna << endl;
        cout << "tahun     : " << P->info.thnBuat << endl;
        P = P->next;
    }
}

address findElm(List L, infotype x) {
    address P = L.First;
    while (P != Nil && P->info.nopol != x.nopol) {
        P = P->next;
    }
```

```
    return P;
}
```

main.cpp

```
#include <iostream>
#include "Doublylist.h"

using namespace std;

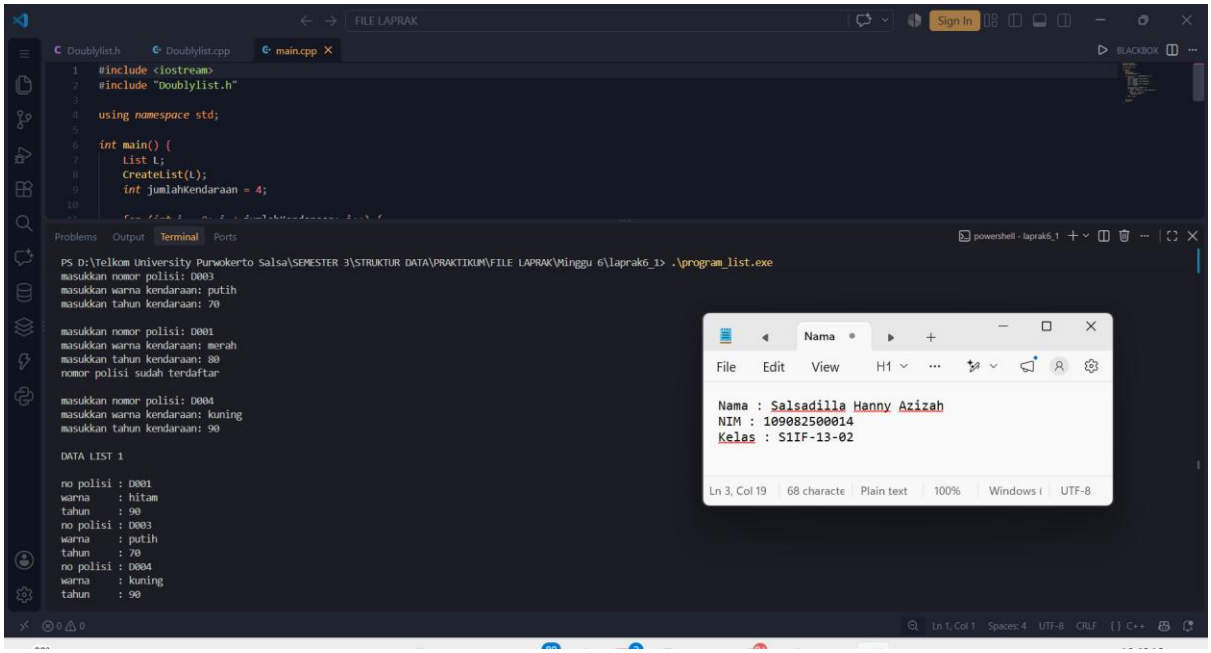
int main() {
    List L;
    CreateList(L);
    int jumlahKendaraan = 4;

    for (int i = 0; i < jumlahKendaraan; i++) {
        infotype k;
        cout << "masukkan nomor polisi: ";
        cin >> k.nopol;
        cout << "masukkan warna kendaraan: ";
        cin >> k.warna;
        cout << "masukkan tahun kendaraan: ";
        cin >> k.thnBuat;

        address ketemu = findElm(L, k);
        if (ketemu != Nil) {
            cout << "nomor polisi sudah terdaftar\n";
        } else {
            address P = alokasi(k);
            insertLast(L, P);
        }
        cout << endl;
    }

    printInfo(L);
    return 0;
}
```

Screenshoot program



Deskripsi program

Program ini dibuat untuk mengimplementasikan Abstract Data Type (ADT) pada struktur Doubly Linked List statik untuk mengelola rekaman data kendaraan. Kode dipecah menjadi tiga file utama untuk memisahkan antara deklarasi tipe/prototipe (Doublylist.h), realisasi bodi fungsi (Doublylist.cpp), dan pengujian (main.cpp). Alur kerjanya difokuskan pada pembentukan list ganda yang kosong, penerimaan input dari terminal, verifikasi pencarian agar tidak terjadi duplikasi nomor polisi, dan penyambungan simpul (node) baru ke posisi paling ujung belakang menggunakan metode insertLast.

Penjelasan kode program:

- **struct ElmList =** Mendeklarasikan bentuk simpul yang memuat variabel info untuk menyimpan data kendaraan, serta dua buah pointer utama: next yang menunjuk ke simpul sesudahnya dan prev yang menunjuk ke simpul sebelumnya.
- **void insertLast(List &L, address P)** = Prosedur penyisipan di posisi akhir. Jika list masih kosong, simpul baru otomatis diatur menjadi elemen First sekaligus Last. Apabila list sudah memiliki isi, pointer prev pada simpul baru dihubungkan ke L.Last, lalu pointer next pada elemen ujung sebelumnya dialihkan menuju simpul baru, dan terakhir tanda L.Last dipindah ke simpul P tersebut.
- **Pengecekan Duplikasi** = Sebelum mengeksekusi insertLast, fungsi utama akan melempar data ke dalam rutin penelusuran findElm. Jika fungsi tidak mengembalikan Nil (artinya data ditemukan), program memunculkan notifikasi peringatan dan melewati proses alokasi simpul.

2. Soal 2

main.cpp

```
#include <iostream>
#include "Doublylist.h"

using namespace std;

int main() {
    List L;
    CreateList(L);
    int jumlahKendaraan = 4;

    // --- 1. Proses Input Data Kendaraan ---
    for (int i = 0; i < jumlahKendaraan; i++) {
        infotype k;
        cout << "masukkan nomor polisi: "; cin >> k.nopol;
        cout << "masukkan warna kendaraan: "; cin >> k.warna;
        cout << "masukkan tahun kendaraan: "; cin >>
k.thnBuat;

        if (findElm(L, k) != Nil) cout << "nomor polisi sudah
terdaftar\n";
        else insertLast(L, alokasi(k));
        cout << endl;
    }
    printInfo(L);
    cout << "-----------------------------------------\n";

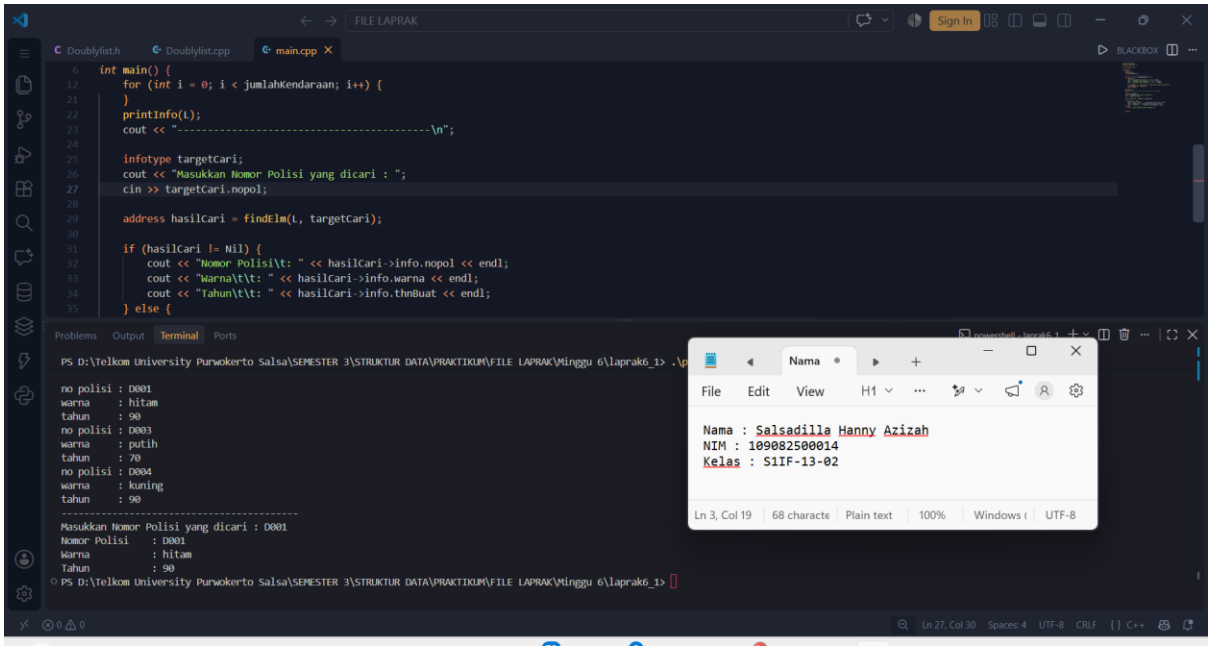
    infotype targetCari;
    cout << "Masukkan Nomor Polisi yang dicari : ";
    cin >> targetCari.nopol;

    address hasilCari = findElm(L, targetCari);

    if (hasilCari != Nil) {
        cout << "Nomor Polisi\t: " << hasilCari->info.nopol
<< endl;
        cout << "Warna\t\t: " << hasilCari->info.warna <<
endl;
        cout << "Tahun\t\t: " << hasilCari->info.thnBuat <<
endl;
    } else {
        cout << "Data kendaraan tidak ditemukan.\n";
    }

    return 0;
}
```

Screenshoot program



Deskripsi program

Program ini merupakan kelanjutan dari kode penginputan data sebelumnya dengan menitikberatkan pada implementasi operasi pencarian letak simpul (searching). Program mengeksekusi penelusuran secara sekuensial dari pangkal awal list untuk mendeteksi lokasi memori kendaraan yang memiliki atribut nomor polisi "D001". Begitu simpul target berhasil dikunci letaknya, program membongkar isi dari simpul tersebut dan mencetak seluruh rincian informasi atributnya (nomor polisi, warna, dan tahun buat) ke layar monitor.

Penjelasan kode program:

- **address findElm(List L, infotype x)** = Fungsi ini didesain sebagai mesin pencari yang mengembalikan alamat letak memori (address). Pointer pembantu P akan berjalan maju mulai dari titik L.First dengan perintah P = P->next, dan perulangannya akan terus digulirkan selama ujung rantai belum tersentuh (P != Nil) dan data nomor polisinya belum identik dengan kata kunci yang ditargetkan.
- **if (hasilCari != Nil)** = Sebuah blok percabangan pengaman di dalam fungsi main untuk mengecek hasil balikan dari proses pencarian. Jika pointer terbukti berisi (tidak kosong), artinya data sah berada di dalam list sehingga instruksi pencetakan cout dapat diakses tanpa menghasilkan galat.

3. Soal 3

Doublylist.h

```
#ifndef DOUBLYLIST_H_INCLUDED
#define DOUBLYLIST_H_INCLUDED
#include <iostream>
#include <string>
```

```
#define Nil NULL
using namespace std;

struct kendaraan {
    string nopol;
    string warna;
    int thnBuat;
};

typedef kendaraan infotype;
typedef struct ElmList *address;

struct ElmList {
    infotype info;
    address next;
    address prev;
};

struct List {
    address First;
    address Last;
};

void CreateList(List &L);
address alokasi(infotype x);
void dealokasi(address &P);
void printInfo(List L);
void insertLast(List &L, address P);
address findElm(List L, infotype x);

void deleteFirst(List &L, address &P);
void deleteLast(List &L, address &P);
void deleteAfter(address Prec, address &P);

#endif
```

Doublylist.cpp

```
#include "Doublylist.h"

void CreateList(List &L) {
    L.First = Nil;
    L.Last = Nil;
}

address alokasi(infotype x) {
    address P = new ElmList;
```

```
    if (P != Nil) {
        P->info = x;
        P->next = Nil;
        P->prev = Nil;
    }
    return P;
}

void dealokasi(address &P) {
    delete P;
    P = Nil;
}

void insertLast(List &L, address P) {
    if (L.First == Nil) {
        L.First = P;
        L.Last = P;
    } else {
        P->prev = L.Last;
        L.Last->next = P;
        L.Last = P;
    }
}

void printInfo(List L) {
    address P = L.First;
    cout << "DATA LIST 1\n" << endl;
    while (P != Nil) {
        cout << "no polisi : " << P->info.nopol << endl;
        cout << "warna     : " << P->info.warna << endl;
        cout << "tahun     : " << P->info.thnBuat << endl;
        P = P->next;
    }
}

address findElm(List L, infotype x) {
    address P = L.First;
    while (P != Nil && P->info.nopol != x.nopol) {
        P = P->next;
    }
    return P;
}

void deleteFirst(List &L, address &P) {
    P = L.First;
    if (L.First != L.Last) {
        L.First = P->next;
        L.First->prev = Nil;
```

```
            P->next = Nil;
        } else {
            L.First = Nil;
            L.Last = Nil;
        }
    }

    void deleteLast(List &L, address &P) {
        P = L.Last;
        if (L.First != L.Last) {
            L.Last = P->prev;
            L.Last->next = Nil;
            P->prev = Nil;
        } else {
            L.First = Nil;
            L.Last = Nil;
        }
    }

    void deleteAfter(address Prec, address &P) {
        P = Prec->next;
        if (P != Nil) {
            Prec->next = P->next;
            if (P->next != Nil) {
                P->next->prev = Prec;
            }
            P->next = Nil;
            P->prev = Nil;
        }
    }
```

main.cpp

```
    #include <iostream>
    #include "Doublylist.h"

    using namespace std;

    int main() {
        List L;
        CreateList(L);
        int jumlahKendaraan = 4;

        for (int i = 0; i < jumlahKendaraan; i++) {
            infotype k;
            cout << "masukkan nomor polisi: "; cin >> k.nopol;
            cout << "masukkan warna kendaraan: "; cin >> k.warna;
```

```
        cout << "masukkan tahun kendaraan: "; cin >>
k.thnBuat;

        if (findElm(L, k) != Nil) cout << "nomor polisi sudah
terdaftar\n";
        else insertLast(L, alokasi(k));
        cout << endl;
    }
    printInfo(L);
    cout << "----------------------------------------\n";

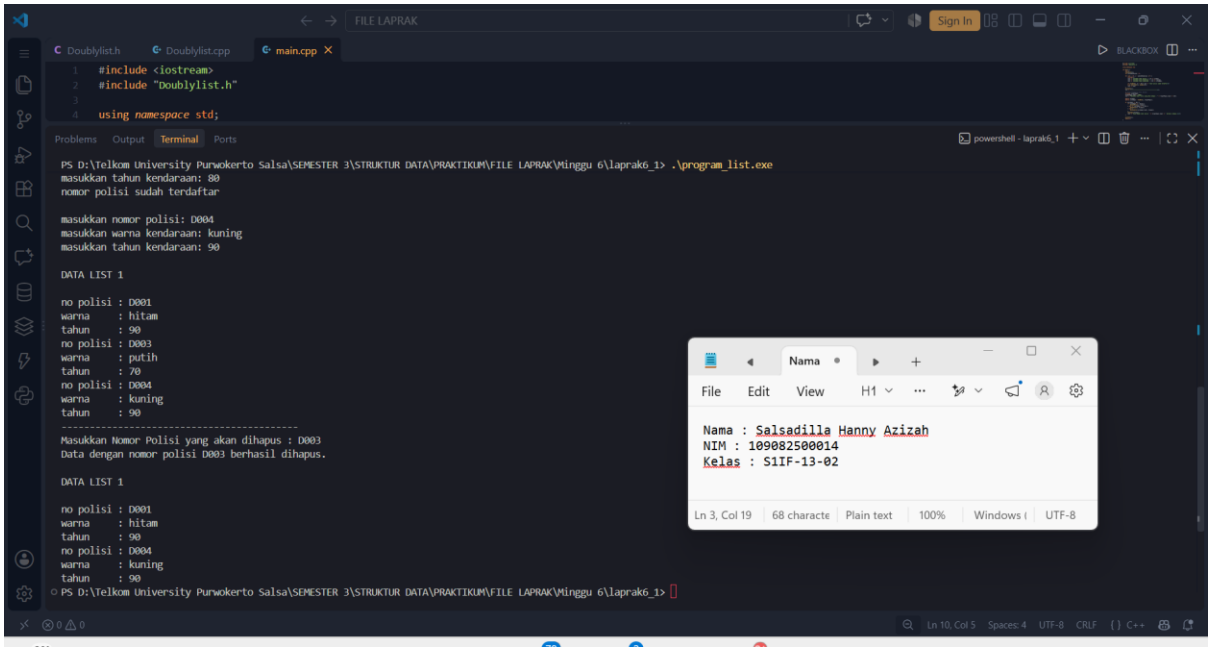
    infotype targetHapus;
    targetHapus.nopol = "D003";
    cout << "Masukkan Nomor Polisi yang akan dihapus : " <<
targetHapus.nopol << endl;

    address P_hapus;
    address cariHapus = findElm(L, targetHapus);

    if (cariHapus != Nil) {
        if (cariHapus == L.First) {
            deleteFirst(L, P_hapus);
        } else if (cariHapus == L.Last) {
            deleteLast(L, P_hapus);
        } else {
            deleteAfter(cariHapus->prev, P_hapus);
        }
        dealokasi(P_hapus);
        cout << "Data dengan nomor polisi " <<
targetHapus.nopol << " berhasil dihapus.\n\n";
    }

    printInfo(L);
    return 0;
}
```

Screenshoot program



Deskripsi program

Program tahap pamungkas ini dirangkai untuk mempertunjukkan keunggulan dan fleksibilitas pengaitan memori dua arah pada Doubly Linked List lewat operasi pelepasan simpul (delete). Alur kerjanya dimulai dari pelacakan letak elemen yang memegang nomor polisi sasaran (D003). Setelah kedudukannya dipastikan, algoritma akan mengevaluasi posisinya apakah menjabat sebagai simpul paling depan, simpul paling belakang, atau terselip di area tenga untuk memilih dan mengeksekusi metode pencabutan yang paling relevan. Begitu pemutusan rantai diselesaikan dengan aman, bekas memori simpul tersebut akan dikosongkan.

Penjelasan kode program:

- **void deleteFirst / deleteLast =** Prosedur yang merawat pencabutan simpul yang bertengger di area pinggiran. deleteFirst melakukan intervensi dengan mencopot label First, memindahkannya ke simpul urutan kedua, dan menetapkan pointer mundurnya (prev) agar terputus ke arah Nil. Sebaliknya, deleteLast mengadopsi mekanisme serupa di urutan ekor dengan mengatrol mundur posisi Last ke arah dalam.
- **void deleteAfter(address Prec, address &P)** = Prosedur pembedahan memori yang bertugas menyingkirkan elemen di kawasan tengah. Dua manipulasi krusial dijalankan menghubungkan pointer next elemen penyangga bagian depan (Prec) ke sisi belakang elemen target, sekaligus merajut kawat prev elemen yang berada di seberangnya untuk melompat kembali mengarah utuh pada Prec.
- **dealokasi(P_hapus) =** Perintah penutup dan rutinitas bersih-bersih (garbage cleaning) yang amat vital untuk melenyapkan kepingan data dari memori secara permanen demi menangkal efek kebocoran RAM (memory leak).